Zero-Dependency MQTT Broker

High-Performance Event-Driven Messaging in C — Mid-Semester Review

Team Members:

Pranav Bhat (231IT049)
Pranav Vinodh (231IT050)
Rijul Raman (231IT056)

Course: IT365 Advanced Computer Networks
Department of Information Technology
National Institute of Technology Karnataka (NITK), Surathkal

October 2026

Presentation Outline

1. Motivation & Philosophy
2. Problem Statement
3. Project Objectives & 3-Phase Division
4. Phase 1 Implementation: Architecture
5. Phase 1 Deep Dive: Core Components
6. Verification & Test Suite
7. Upcoming Roadmap: Phases 2 & 3
8. Summary & References

Project Motivation & Philosophy

The Problem with Modern Brokers

- Commercial brokers (Mosquitto, EMQX, HiveMQ) feature **large third-party dependencies**.
- Rely on complex abstractions (‘libuv‘, ‘libevent‘, ‘Boost.Asio‘) obscuring network primitives.
- High memory consumption, making deployment difficult on constrained embedded hardware.

Our First-Principles Approach

- **Zero External Dependencies:** Pure standard C (C11) using native POSIX sockets and Linux `epoll`.
- **Addressing the C10K Problem:** Asynchronous Reactor pattern using level-triggered `epoll` with $O(1)$ event scalability.
- **Wire-Level Conformance:** Handcrafted bitwise decoders enforcing OASIS MQTT 3.1.1 specification rules.

Project Problem Statement

Core Technical Problem Formulation

Design and implement an ultra-lightweight, high-performance MQTT 3.1.1 message broker from first principles in C, without external networking or parsing libraries, while rigorously addressing fundamental computer systems and network challenges.

Four Primary Technical Challenges:

1. **TCP Stream Defragmentation:** TCP is an unstructured byte stream. The broker must frame variable-length packets under arbitrary fragmentation, partial reads, and packet coalescence.
2. **Wire-Level Bitwise Decoding:** Strict decoding of bit-shifted headers, variable-byte remaining lengths (1–4 bytes), and spec-mandated flag validation (OASIS Table 2.2).
3. **Scalable Concurrency (C10K):** Eliminating blocking operations and heavy thread-per-connection architectures via an asynchronous non-blocking event reactor.
4. **Hierarchical Topic Wildcards:** Resolving subscriptions ('+' single-level, '#' multi-level) efficiently without costly $O(N)$ linear scans.

Project Objectives

1. **Native Linux Non-Blocking Network Core:** Implement an asynchronous TCP server using Linux `epoll` with non-blocking sockets, disabling Nagle's algorithm (`TCP_NODELAY`) for minimal latency.
2. **Dynamic TCP Stream Framing Engine:** Develop an accumulating, compacting linear stream buffer (`byte_buffer_t`) capable of reliably reassembling packets under pathological byte-by-byte fragmentation.
3. **Strict Bitwise MQTT 3.1.1 Parser & Serializer:** Build standard-compliant wire decoders for control packets (`CONNECT`, `PUBLISH`, `SUBSCRIBE`, `PINGREQ`, `DISCONNECT`) and binary response generators.
4. **Hierarchical Topic Radix Tree (Topic Trie):** Implement an $O(K)$ hierarchical prefix tree supporting single-level ('+') and multi-level ('#') wildcard subscriptions.
5. **Resilient Session & QoS Management:** Support Clean/Persistent session persistence, QoS 0/1 packet identifier tracking, keep-alive watchdog timers ($1.5\times$ interval), and Last Will and Testament (LWT).

3-Phase Engineering Roadmap

Phase 1: Foundation

[COMPLETED]

- Non-blocking TCP Server
- Linux `epoll` Reactor
- Stream accumulator buffer
- Bitwise MQTT 3.1.1 parser
- Binary response serializer
- Full Automated Test Suite

Phase 2: Handshake & Routing

[CURRENT FOCUS]

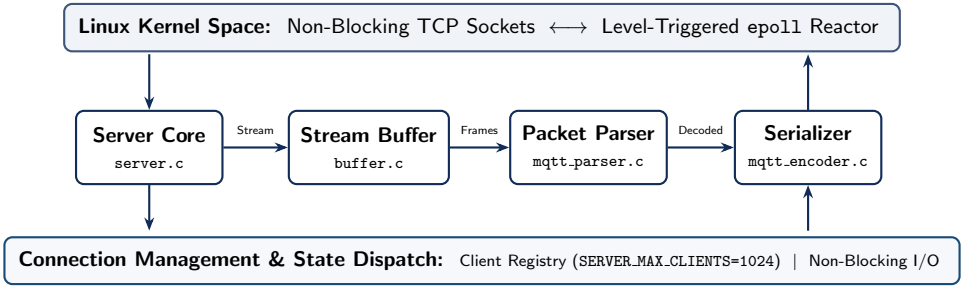
- CONNECT / CONNACK logic
- Clean vs Persistent sessions
- Client session registry
- Topic Radix Tree (Trie)
- Wildcard matcher (‘+’, ‘#’)
- SUBSCRIBE / SUBACK flow

Phase 3: Fan-Out & Resilience

[PLANNED]

- PUBLISH payload fan-out
- Non-blocking TX buffer queue
- QoS 0 & QoS 1 flow (PUBACK)
- Subscriber deduplication
- Keep-alive timeout watchdog
- Last Will and Testament

Phase 1 System Architecture



- **Decoupled Pipeline:** Socket notification → stream buffer defragmentation → wire decoding → response encoding → non-blocking socket writes.

Component 1: Non-Blocking TCP Server Core

Socket Options & Network Setup

- `SO_REUSEADDR` & `SO_REUSEPORT`: Instant broker restart without `TIME_WAIT` lockup.
- `TCP_NODELAY`: Disables Nagle's algorithm to eliminate 40ms delayed-ACK latencies.
- `O_NONBLOCK`: Applied to listening and client sockets via `fcntl()`.
- `SIGPIPE` Ignored: Prevents broker exit on broken peer connections.

Linux `epoll` Reactor Execution

- **Instance Creation:** `epoll_create1(EPOLL_CLOEXEC)` provides $O(1)$ event notifications.
- **Non-Blocking Accept Loop:** Uses `accept4()` in a loop until `EAGAIN` to drain burst connection storms.
- **Level-Triggered Multiplexing:**
  - Master socket registers `EPOLLIN`.
  - Client sockets register `EPOLLIN | EPOLLRDHUP | EPOLLERR`.
  - Dynamic `EPOLLOUT` on partial socket writes.

Component 2: Stream Buffer & Defragmentation

The TCP Framing Challenge

- TCP is an unstructured byte stream.
- Packets can arrive fragmented across reads.
- Multiple packets can arrive concatenated.
- Frames may arrive 1 byte at a time.

Bounded Data Structure

```
typedef struct {
    uint8_t *data;          // Buffer pointer
    size_t   size;          // Active valid bytes
    size_t   capacity;      // Allocated bytes
    size_t   max_capacity; // 512 KB DoS ceiling
} byte_buffer_t;
```

Buffer Compaction & Growth

- **Geometric Capacity Doubling:** `byte_buffer_reserve()` doubles allocated capacity up to `max_capacity` (512 KB).
- **Linear Compaction via `memmove()`:** When a packet is parsed, `byte_buffer_drain()` slides remaining bytes to index 0:

$$\text{offset } 0 \longleftarrow \text{data}[P \ldots N]$$

- **Zero Truncation:** Partial frames return `MQTT_NEED_MORE_DATA` without consuming bytes.

Component 3: Bitwise OASIS MQTT 3.1.1 Decoder

Fixed Header Byte 0 Validation

- Type $= (\text{Byte}_0 \gg 4)$ & 0x0F
- Flags $= \text{Byte}_0$ & 0x0F
- **OASIS Table 2.2 Spec Enforcement:**
  - `CONNECT`, `CONNACK`, `PUBACK`, `PINGREQ`, `DISCONNECT`: **MUST** be `0000`.
  - `SUBSCRIBE`, `PUBREL`: **MUST** be `0010`.
  - Flag violations trigger immediate drop.

PUBLISH Flags & Constraints

- DUP: $(\text{Flags} \gg 3)$ & 0x01
- QoS: $(\text{Flags} \gg 1)$ & 0x03
- RETAIN: Flags & 0x01
- Strict validation: Rejects QoS 3, or QoS 0 with DUP $= 1$.
- Wildcard check: Rejects '+' and '#' in PUBLISH topics.

Variable-Byte Remaining Length Decoding (Section 2.2.3)

$$\text{Length} = \sum_{i=0}^{m \leq 3} (\text{byte}_i \ \& \ 0\text{x}7\text{F}) \times 128^i$$ (Encodes 0 to 256 MB in 1–4 bytes)

Component 4: Implemented Decoders & Encoders

Wire Decoders (`src/mqtt_parser.c`)

- **CONNECT:** Decodes protocol name ("MQTT"), level (0x04), CleanSession, KeepAlive, ClientID, Will, and credentials.
- **PUBLISH:** Extracts topic name, 16-bit PacketID (QoS > 0), and raw payload pointers.
- **SUBSCRIBE:** Extracts 16-bit PacketID, topic filters, and requested QoS levels (0, 1, 2).
- **PINGREQ / DISCONNECT:** Validates remaining length is exactly 0.

Binary Encoders (`src/mqtt_encoder.c`)

- **CONNACK (4 bytes):**
  `[0x20, 0x02, session_present, rc]`
- **PINGRESP (2 bytes):**
  `[0xD0, 0x00]`
- **PUBACK (4 bytes):**
  `[0x40, 0x02, (pid>>8), (pid&0xFF)]`
- **SUBACK (Variable length):**
  Header 0x90, remaining length, 16-bit PacketID, and granted QoS array.

Phase 1 Verification: Test Suite Results

Three-Tier Testing Pyramid

1. **Buffer Tests (`test_buffer.c`):** Dynamic growth, compaction, and memory bounds.
2. **Parser Tests (`test_parser.c`):** Variable integers (1–4 bytes), Table 2.2 flags, 1-byte chunk streams, pipelining.
3. **E2E Tests (`test_server_e2e.py`):** Real TCP sockets, handshakes, heartbeats, QoS 1 PUBACK, 30 concurrent sockets.

Live Test Execution Log

```
>>> RUNNING BUFFER TESTS <<<
  -> PASS: test_buffer_basic
  -> PASS: test_buffer_growth
ALL BUFFER TESTS PASSED!

>>> RUNNING PARSER & FRAMING TESTS <<<
  -> PASS: test_remaining_length_encoding_decoding
  -> PASS: test_fixed_header_flags
  -> PASS: test_connect_packet_parsing
  -> PASS: test_publish_packet_parsing
  -> PASS: test_subscribe_packet_parsing
  -> PASS: test_stream_defragmentation (1-byte chunk)
  -> PASS: test_packet_pipelining (concatenated)
ALL MQTT PARSER TESTS PASSED!

>>> RUNNING E2E INTEGRATION TESTS <<<
  -> CONNECT handshake ..................... [PASS]
  -> PINGREQ heartbeat ..................... [PASS]
  -> PUBLISH QoS 1 / PUBACK ................ [PASS]
  -> SUBSCRIBE / SUBACK .................... [PASS]
  -> Stream Defragmentation (delays) ..... [PASS]
  -> 30 Concurrent Client Sockets ........ [PASS]
ALL PHASE 1 TESTS PASSED (100% OK)
```

What Remains to be Implemented: Phase 2

1. Session State Machine & Registry

- **Lifecycle States:** Transition flows `NEW` → `CONNECTED` → `CLOSED`.
- **Duplicate ClientID:** Terminate older socket cleanly per MQTT-3.1.4-2.
- **Session Persistence:**
  - Clean=1: Destroy state on disconnect.
  - Clean=0: Retain subscriptions and unacknowledged QoS 1 offline queue.
- Thread-safe registry protected by `pthread_mutex_t`.

2. Topic Radix Tree (Topic Trie)

- Multi-way prefix tree splitting topic levels on ''/''.
- Guarantees $O(K)$ lookup invariant to subscription volume.
- **Wildcard Matching:**
  - Single-level '+': Matches one level (e.g., `sensors/+/temp`).
  - Multi-level '#': Matches all subordinate suffix levels (e.g., `sensors/#`).
- Dynamic `SUBSCRIBE` / `UNSUBSCRIBE` operations.

What Remains to be Implemented: Phase 3

1. Message Fan-Out & QoS 1 Flow

- **Trie Traversal on PUBLISH:** Matches published topic against exact and wildcard subscriber nodes.
- **Subscriber Deduplication:** Prevents duplicate message delivery per MQTT-3.3.5-1.
- **Non-blocking TX Queue:** Per-client `tx_buffer` with level-triggered `EPOLLOUT`.
- **QoS 1 In-flight Tracking:** Retransmission and packet ID matching.

2. Reliability, Watchdogs & Teardown

- **Keep-Alive Watchdog Timer:** Monitored via epoll tick. Terminate socket if silent $> 1.5 \times$ keep_alive.
- **Last Will and Testament (LWT):** Automatically publish stored Will message upon ungraceful connection drop.
- **Graceful Disconnect:** Clean teardown on `DISCONNECT` packet without triggering Will.

Summary & Mid-Semester Status

Key Accomplishments (Phase 1)

- Event-driven TCP network core in pure C11 using native Linux `epoll`.
- Solved TCP stream defragmentation with a self-compacting linear byte buffer.
- Handcrafted bitwise OASIS MQTT 3.1.1 packet parser with 100% test coverage.
- Zero external runtime dependencies utilized.

Next Immediate Milestones

- Implement Topic Radix Tree (Trie) for wildcard topic filtering.
- Complete CONNECT/CONNACK session state machine.
- Message fan-out and QoS 1 delivery across active client sockets.

References



A. Banks and R. Gupta,
MQTT Version 3.1.1,
OASIS Standard, Oct. 2014.



W. R. Stevens, B. Fenner, and A. M. Rudoff,
UNIX Network Programming: The Sockets Networking API, vol. 1,
Addison-Wesley Professional, 3rd ed., 2003.



D. R. Morrison,
"PATRICIA—Practical Algorithm To Retrieve Information Coded in Alphanumeric,"
Journal of the ACM, vol. 15, no. 4, pp. 514–534, 1968.



D. Kegel,
"The C10K problem,"
Online: http://www.kegel.com/c10k.html, 1999.

Thank You!

Questions & Discussion

GitHub Repository:
`https://github.com/PranavBhatP/zero-dep-mqtt-broker`